

WTB 故障模式详解

文档目的： 解释 WTB (Workflow Test Bench) 如何解决 AI/ML 工作流中的常见故障模式，包含详细机制、SDK 使用方法和成本分析。

版本： 1.8 (2026-02-07)

执行摘要

WTB 解决了 AI/ML 工作流中的 **7 种关键故障模式**。对于每种故障模式，本文档说明：

- 问题** - 可能出错的情况
- 解决方案** - WTB 如何解决
 - 原理** - 底层机制
 - SDK 用法** - 实际使用方式
- 成本分析** - 开销与收益权衡

故障模式 1：幽灵答案（LLM 响应非确定性）

问题

当 LLM 调用失败并重试时，每次重试可能产生**不同的答案**，原因包括：

- 温度 > 0（随机采样）
- 调用之间模型更新
- 重试时提示格式不同

影响： 临床医生可能收到不同的治疗建议，取决于哪次重试成功。

SOTP（现有实践）：

重试1: "建议使用药物A 500mg" ← 失败，丢弃

重试2: "建议使用药物B 250mg" ← 成功，返回

结果：答案取决于重试时机，而非临床证据。
可重现性：35.9%

解决方案

原理：幂等键

WTB 使用**执行范围的幂等键**确保重试间响应一致：

```
# 内部工作原理
class IdempotentLLMService:
    def call(self, prompt: str, execution_id: str) -> str:
        # 根据内容 + 执行上下文生成幂等键
        key = hash(f"{prompt}:{execution_id}")

        # 检查是否见过这个完全相同的调用
        if key in self._cache:
            return self._cache[key] # 返回相同答案

        # 首次调用 - 发起真实 API 请求
        response = self._llm.generate(prompt)
        self._cache[key] = response
        return response
```

关键洞察：**execution_id 绑定到 workflow 运行**，而非重试尝试。同一执行中的所有重试获得相同答案。

SDK 用法

```
from wtb.sdk import WorkflowTestBench

# 创建启用幂等性的测试台（默认启用）
bench = WorkflowTestBench.create(
    db_path="data/wtb.db",
    enable_idempotency=True, # 默认: True
)

# 运行 workflow - 重试自动使用相同的 execution_id
result = bench.run_workflow(
    workflow=my_workflow,
    initial_state={"query": "糖尿病用什么药?"},
)

# 即使内部 LLM 调用重试，答案也是确定的
print(result.state["answer"]) # 相同输入始终得到相同答案
```

一行代码： 只需使用 `WorkflowTestBench.create()` - 幂等性默认启用。

成本分析

成本项	SOTP	WTB	WTB 开销
API 调用（3次重试 × 5文档）	15 次真实调用	5 次真实 + 10 次缓存	哈希计算： ~0.01ms
API 费用	\$0.015	\$0.005	-
缓存存储	无	~1KB/调用	SQLite 写入： ~0.5ms
总 API 成本	基准的 2.21 倍	基准的 1.00 倍	~0.5ms/调用

净收益： API 成本降低 54.7%，每次调用开销 ~0.5ms。

故障模式 2：幽灵引用（孤儿向量）

问题

当向多个存储写入（SQL + VectorDB）时，写入间的崩溃会创建**孤儿记录**：

事务：

- 1. 写入 SQL: `citation_id=123, text="研究表明..."` ✓
- 2. 写入 VectorDB: `vector_id=123, embedding=[...]` ← 崩溃

结果：SQL 有 `citation_id=123`，但 VectorDB 没有匹配的向量。
无法通过语义搜索检索引用。
审计追踪中断。

解决方案

原理：补偿事务（Saga 模式）

WTB 实现了带补偿事务的 **Saga 模式**：

```

# 内部工作原理
class UnitOfWork:
    def __enter__(self):
        self._operations = []
        self._compensations = []

    def add_sql(self, record):
        self._operations.append(("sql", record))
        # 注册补偿（反向操作）
        self._compensations.append(lambda: self._sql.delete(record.id))

    def add_vector(self, vector):
        self._operations.append(("vector", vector))
        self._compensations.append(lambda: self._vectordb.delete(vector.id))

    def commit(self):
        try:
            for op_type, data in self._operations:
                self._execute(op_type, data)
        except Exception:
            # 按相反顺序回滚
            for compensation in reversed(self._compensations):
                compensation()
            raise

```

关键洞察： 每次写入都注册其"撤销"操作。失败时，WTB 按相反顺序执行补偿。

SDK 用法

```

from wtb.sdk import WorkflowTestBench

bench = WorkflowTestBench.create(db_path="data/wtb.db")

# 使用工作单元进行多存储操作
with bench.unit_of_work() as uow:
    # 添加到 SQL
    uow.citations.add(Citation(id="123", text="研究表明..."))

    # 添加到 VectorDB
    uow.vectors.add(Vector(id="123", embedding=[0.1, 0.2, ...]))

# 提交 - 如果任一失败，两者都回滚
uow.commit()

# 或使用高级 API（隐式 UoW）
bench.add_citation_with_embedding(
    citation_id="123",
    text="研究表明...",
    embedding=[0.1, 0.2, ...],
) # 原子性：两者都成功或都失败

```

一行代码：用 `with bench.unit_of_work() as uow` 包装多存储操作。

成本分析

成本项	SOTP	WTB	WTB 开销
写操作	2 次写入	2 次写入 + 2 次补偿注册	~0.1ms
崩溃恢复	手动清理	自动回滚	失败时 ~10ms
审计完整性	45% 孤儿率	0% 孤儿率	-
总开销	N/A	~0.1ms/操作	失败时 ~10ms

净收益： 100% 数据完整性，每次操作开销 ~0.1ms。

故障模式 3：协议乱序（竞态条件）

问题

在并发执行中，步骤可能乱序完成：

临床协议（必须顺序执行）：

- 1. 检查过敏
- 2. 验证药物相互作用
- 3. 实施化疗

并发执行（SOTP）：

- 线程1：检查过敏（100ms 延迟）
- 线程2：验证相互作用（50ms 延迟）
- 线程3：实施化疗（10ms 延迟）← 最先完成！

结果：化疗在过敏检查前执行。

解决方案

原理：LangGraph 检查点屏障

WTB 使用 LangGraph 的 StateGraph 配合检查点作为执行屏障：

```

# 内部工作原理
builder = StateGraph(ProtocolState)
builder.add_node("check_allergies", check_allergies_node)
builder.add_node("verify_interactions", verify_interactions_node)
builder.add_node("administer_chemo", administer_chemo_node)

# 顺序边强制执行顺序
builder.add_edge(START, "check_allergies")
builder.add_edge("check_allergies", "verify_interactions") # 必须等待
builder.add_edge("verify_interactions", "administer_chemo") # 必须等待
builder.add_edge("administer_chemo", END)

# 使用检查点器编译 - 每条边创建一个屏障
graph = builder.compile(checkpointer=SqliteSaver.from_conn_string(db_path))

```

关键洞察： 图中的每条边都是**检查点屏障**。下一个节点在前一个节点的检查点持久化之前无法启动。

SDK 用法

```

from wtb.sdk import WorkflowTestBench, WorkflowBuilder

bench = WorkflowTestBench.create(db_path="data/wtb.db")

# 定义具有明确顺序的工作流
workflow = (
    WorkflowBuilder("clinical_protocol")
        .add_node("check_allergies", check_allergies_fn)
        .add_node("verify_interactions", verify_interactions_fn)
        .add_node("administer_chemo", administer_chemo_fn)
        .add_sequential_edges([ # 强制顺序
            "check_allergies",
            "verify_interactions",
            "administer_chemo",
        ])
        .build()
)

# 运行 - 顺序有保证
result = bench.run_workflow(workflow, initial_state)
# check_allergies 始终在 verify_interactions 之前完成

```

一行代码： 使用 `.add_sequential_edges()` 强制执行步骤顺序。

成本分析

成本项	SOTP	WTB	WTB 开销
执行模型	并发（竞态）	顺序（有序）	-
检查点写入	无	1 次/步骤	~1-2ms/步骤
顺序保证	0%	100%	-
总开销	N/A	~1-2ms/步骤	Kendall's tau: 1.0

净收益： 100% 顺序保证，每步骤开销 ~1-2ms。

故障模式 4：僵尸指南（陈旧缓存）

问题

当数据更新时，缓存副本变得**陈旧且危险**：

时间线：
T0: 缓存指南: "二甲双胍最大剂量: 2000mg"
T1: FDA 发布安全警报: "eGFR < 45 时最大剂量: 1000mg"
T2: 查询命中陈旧缓存: "最大剂量: 2000mg" ← 危险

eGFR=35 的患者收到 2000mg 而非安全的 1000mg。

解决方案

原理：缓存失效的发件箱模式

WTB 使用**发件箱模式**确保缓存失效与数据更新原子进行：

```
# 内部工作原理
class GuidelineCacheWithOutbox:
    def update_guideline(self, drug: str, new_data: dict, uow: UnitOfWork):
        # 1. 更新后端
        self._backend.update(drug, new_data)

        # 2. 在同一事务中排队失效事件
        outbox_event = OutboxEvent.create(
            event_type=OutboxEventType.CACHE_INVALIDATE,
            aggregate_id=drug,
```

```
        payload={"version": new_data["version"]},
    )
    uow.outbox.add(outbox_event)

    # 3. 使缓存失效
    self._cache.invalidate(drug)

    # 4. 原子提交 - 更新和失效一起
    uow.commit()
```

关键洞察： 失效事件在与数据更新相同的事务中写入发件箱表。即使进程在提交后崩溃，发件箱处理器最终也会使缓存失效。

SDK 用法

```
from wtb.sdk import WorkflowTestBench

bench = WorkflowTestBench.create(
    db_path="data/wtb.db",
    enable_outbox=True, # 默认: True
)

# 带自动缓存失效的更新
with bench.unit_of_work() as uow:
    # 更新后端数据
    uow.guidelines.update("metformin", {
        "max_dose": "1000mg",
        "condition": "eGFR < 45",
        "version": "v2.1-SAFETY",
    })

    # 通过发件箱自动使缓存失效
    uow.commit()

# 或使用高级 API
bench.update_guideline(
    drug="metformin",
    data={"max_dose": "1000mg", ...},
    invalidate_cache=True, # 默认: True
)
```

一行代码： 使用 `uow.commit()` - 缓存失效通过发件箱自动完成。

成本分析

成本项	SOTP	WTB	WTB 开销
陈旧读取率	73%	20%（已检测）	-
发件箱事件写入	无	1 次/更新	~0.5ms
缓存失效	手动	自动	~0.1ms
安全评分	0.21	0.79	-
总开销	N/A	~0.6ms/更新	安全性提升 3.8 倍

净收益： 安全性提升 3.8 倍，每次更新开销 ~0.6ms。

故障模式 5： 队列重复（重复计数）

问题

处理批次时，崩溃后重启可能重新处理已计数的项目：

处理 100 名患者：

患者 1-50： 已处理，插入 VectorDB ✓

患者 51： 崩溃

SOTP 重启（从头开始）：

患者 1-50： 重新处理，重新插入 ← 重复！

患者 51-100： 处理，插入

VectorDB 现有 150 条记录（50 条重复）。

解决方案

原理： 基于检查点的恢复与 ID 跟踪

WTB 在检查点状态中跟踪已处理的 ID，实现精确一次处理：

```
# 内部工作原理
@node
def process_batch(state: BatchState) -> BatchState:
    processed = set(state.get("processed_ids", []))
```

```

for item in state["items"]:
    if item.id in processed:
        continue # 跳过 - 已在检查点中

    # 处理项目
    result = process(item)
    store.insert(item.id, result)

    # 在状态中跟踪（自动检查点）
    processed.add(item.id)

return {"processed_ids": list(processed)}

```

关键洞察： `processed_ids` 集合是检查点状态的一部分。重启时，WTB 加载检查点并跳过已处理的项目。

SDK 用法

```

from wtb.sdk import WorkflowTestBench, BatchRunner

bench = WorkflowTestBench.create(db_path="data/wtb.db")

# 使用 BatchRunner 进行精确一次处理
runner = bench.create_batch_runner(
    checkpoint_interval=10, # 每 10 项检查点
)

# 运行批次 - 重启时自动从检查点恢复
result = runner.run_batch(
    items=patients,
    processor=process_patient,
)

# 或使用图工厂模式进行 Ray 分布式执行
from wtb.sdk import RayBatchTestRunner

ray_runner = RayBatchTestRunner.create(
    db_path="data/wtb.db",
    num_workers=4,
)

# 带检查点支持的分布式批处理
result = ray_runner.run_batch_test(batch_test)

```

一行代码： 使用 `bench.create_batch_runner()` 进行精确一次批处理。

成本分析

成本项	SOTP	WTB	WTB 开销
重复率	50%（中点崩溃）	0%	-
检查点写入	无	1 次/间隔	~1-2ms/检查点
恢复时间	完全重启	从检查点	~5ms 加载检查点
总开销	N/A	~1-2ms/检查点	100% 数据完整性

净收益： 100% 数据完整性，每个检查点间隔开销 ~1-2ms。

故障模式 6：诊断级联（爆炸半径）

问题

当管道中某节点失败时，SOTP 会重启整个管道：

5 节点管道：
节点 1: ✓ (10 秒)
节点 2: ✓ (10 秒)
节点 3: ✗ 失败 (瞬态错误)
节点 4: 未到达
节点 5: 未到达

SOTP 重试：重新运行节点 1、2、3、4、5
爆炸半径：100% (所有节点重新执行)
浪费：20 秒已完成的工作

解决方案

原理：每节点检查点与选择性恢复

WTB 在每个节点边界创建检查点，实现最小重新执行：

```
# 内部工作原理
def run_with_checkpoints(graph, config, initial_state):
    try:
        # 每个节点自动创建检查点
        result = graph.invoke(initial_state, config)
    except Exception:
```

```
# 失败时，获取检查点历史
history = graph.get_state_history(config)
last_good = history[0] # 最近成功的检查点

# 从最后一个好的检查点恢复
result = graph.invoke(
    last_good.values,
    config,
    checkpoint_id=last_good.checkpoint_id,
)

return result
```

关键洞察： LangGraph 的 `invoke()` 在每个节点后创建检查点。失败时，WTB 从最后一个成功的检查点恢复，跳过已完成的节点。

SDK 用法

```
from wtb.sdk import WorkflowTestBench

bench = WorkflowTestBench.create(db_path="data/wtb.db")

# 运行带自动检查点恢复的工作流
result = bench.run_workflow(
    workflow=my_workflow,
    initial_state=initial_state,
    retry_from_checkpoint=True, # 默认: True
)

# 或手动控制回滚
execution = bench.run_workflow(workflow, initial_state)

# 稍后：回滚到特定检查点
rolled_back = bench.rollback(
    execution_id=execution.id,
    checkpoint_id="checkpoint_abc123",
)

# 或从检查点分叉进行探索
forked = bench.fork(
    execution_id=execution.id,
    checkpoint_id="checkpoint_abc123",
    new_state={"exploration_mode": True},
)
```

一行代码： 设置 `retry_from_checkpoint=True`（默认）实现自动最小重新执行。

成本分析

成本项	SOTP	WTB	WTB 开销
爆炸半径（在节点 3/5 失败）	100%	20%	-
重新执行成本	5 倍节点成本	1 倍节点成本	-
检查点写入	无	1 次/节点	~1-2ms/节点
回滚时间	完全重启	加载检查点	~5ms
总开销	N/A	~1-2ms/节点	节省 80% 重新执行

净收益： 节省 80% 重新执行，每节点开销 ~1-2ms。

故障模式 7：实验污染（工作空间泄漏）

问题

在 A/B 测试中，共享文件系统允许变体间数据泄漏：

A/B 测试：

变体 A（治疗组）：写入 /shared/results.json

变体 B（对照组）：读取 /shared/results.json ← 被污染！

对照组看到了治疗组数据，使实验无效。

解决方案

原理：每变体工作空间隔离

WTB 为每个变体创建隔离的工作空间并进行文件跟踪：

```
# 内部工作原理
class WorkspaceManager:
    def create_workspace(self, batch_id: str, variant: str) -> Workspace:
        # 创建隔离目录
        path = self._base / f"batch_{batch_id}" / variant
        path.mkdir(parents=True, exist_ok=True)

        return Workspace(
            workspace_id=f"ws-{uuid4()}",
```

```
        root_path=path,
        file_tracker=SqliteFileTrackingService(path),
    )

    def cleanup_workspace(self, workspace: Workspace, preserve: bool = False):
        if not preserve:
            shutil.rmtree(workspace.root_path)
```

关键洞察： 每个变体在自己的目录中运行。文件跟踪确保无交叉污染，并能回滚文件更改。

SDK 用法

```
from wtb.sdk import RayBatchTestRunner, VariantCombination

# 创建带工作空间隔离的运行器
runner = RayBatchTestRunner.create(
    db_path="data/wtb.db",
    workspace_config={
        "base_path": "workspaces/",
        "isolate_variants": True, # 默认: True
    },
)

# 定义变体
batch_test = BatchTest(
    workflow_id="ab_test",
    variant_combinations=[
        VariantCombination(name="Treatment_A", variants={"model": "gpt-4"}),
        VariantCombination(name="Control_B", variants={"model": "gpt-3.5"}),
    ],
)

# 运行 - 每个变体获得隔离的工作空间
result = runner.run_batch_test(batch_test)

# 工作空间:
#   workspaces/batch_xxx/Treatment_A/  ← 隔离
#   workspaces/batch_xxx/Control_B/    ← 隔离
```

一行代码： 设置 `isolate_variants=True`（默认）实现自动工作空间隔离。

成本分析

成本项	SOTP	WTB	WTB 开销
污染率	100%	0%	-

成本项	SOTP	WTB	WTB 开销
工作空间创建	无	1 次/变体	~10ms
文件跟踪	无	每次文件写入	~0.5ms/文件
清理	手动	自动	~50ms/工作空间
总开销	N/A	~60ms/变体	100% 隔离

净收益： 100% 实验隔离，每变体开销 ~60ms。

回滚和分叉操作

原理：图工厂模式

对于分布式执行（Ray），WTB 使用可导入的图工厂引用而非序列化图：

```
# 内部工作原理
@dataclass
class VariantCombination:
    name: str
    variants: Dict[str, str]
    # 图工厂的可序列化引用
    graph_factory_module: Optional[str] = None # "myapp.workflows"
    graph_factory_name: Optional[str] = None # "create_graph"

    def create_graph(self) -> CompiledStateGraph:
        module = importlib.import_module(self.graph_factory_module)
        factory = getattr(module, self.graph_factory_name)
        return factory()
```

回滚/分叉的 SDK 用法

```
from wtb.sdk import RayBatchTestRunner

runner = RayBatchTestRunner.create(db_path="data/wtb.db")

# 运行批量测试
result = runner.run_batch_test(batch_test)

# 创建回滚/分叉协调器
coordinator = runner.create_rollback_coordinator()
```

```
# 回滚到检查点（LangGraph 操作需要图）
from myapp.workflows import create_my_graph

execution = coordinator.rollback(
    execution_id="exec-123",
    checkpoint_id="cp-456",
    graph=create_my_graph(), # LangGraph 状态适配器需要
)

# 从检查点分叉（非破坏性）
forked = coordinator.fork(
    execution_id="exec-123",
    checkpoint_id="cp-456",
    new_state={"exploration_mode": True},
    graph=create_my_graph(),
)
```

回滚/分叉的成本分析

操作	开销	组件
图工厂导入	~10ms	<code>importlib.import_module()</code>
会话初始化	~2ms	连接到执行的检查点历史
发件箱事件写入	~0.5ms	ACID 审计追踪
检查点加载	~5ms	SQLite 读取
总回滚	~20ms	完整状态恢复
总分叉	~25ms	新执行 + 检查点复制

总体成本摘要

每个解决方案的开销

故障模式	解决方案	开销	收益
幽灵答案	幂等键	~0.5ms/调用	API 成本降低 54.7%
幽灵引用	补偿事务	~0.1ms/操作，失败时 ~10ms	100% 数据完整性
协议乱序	检查点屏障	~1-2ms/步骤	100% 顺序保证

故障模式	解决方案	开销	收益
僵尸指南	发件箱失效	~0.6ms/更新	安全性提升 3.8 倍
队列重复	检查点恢复	~1-2ms/检查点	100% 数据完整性
诊断级联	每节点检查点	~1-2ms/节点	节省 80% 重新执行
实验污染	工作空间隔离	~60ms/变体	100% 实验隔离

何时使用 WTB

场景	推荐	原因
LLM 密集型管道	是	节省 54-67% API 成本
临床/研究 workflow	是	可重现性、审计追踪
长时间运行的任务	是	检查点恢复、最小重新执行
合规要求	是	ACID 事务、发件箱审计
简单单步任务	否	开销不划算
要求亚毫秒延迟	否	~1-2ms 检查点开销

快速开始

```
from wtb.sdk import WorkflowTestBench

# 一行代码获得所有保护
bench = WorkflowTestBench.create(db_path="data/wtb.db")

# 运行具有完整 ACID 合规性的 workflow
result = bench.run_workflow(my_workflow, initial_state)
```